



Опрос про ИИ внутри разработки

Войти



grelikt

40 минут назад

redb 3.3.0: решение уровня энтерпрайз на .NET — своя БД, свой Apache Camel и рантайм с дашбордом (и всё это бесплатно)



Сложный



16 мин



957

.NET*, C#*, Open source*, SQLite*

Тutorial



redb Ecosystem

Серия: redb ecosystem (инженерный анонс 3.3.0)

Когда говорят «энтерпрайз-стек на .NET», обычно имеют в виду зоопарк: база от одного вендора, шина от другого, ORM с миграциями, отдельный оркестратор, ещё что-нибудь для наблюдаемости — и клей между всем этим, который пишешь сам и потом сам же чинишь по ночам.

Мы пошли другим путём и последние полгода собираем это как **одну согласованную экосистему**: типизированное хранилище **redb** поверх Postgres/MSSQL/SQLite, интеграционный движок **redb.Route** (наш ответ Apache Camel под .NET) и рантайм **redb.Tsak** с дашбордом, hot-reload и кластером. Три слоя, один код, один стиль.

Сегодня вышла версия **3.3.0** — синхронный бамп всей экосистемы. Это не «добавили пару фич»: это релиз, в котором мы **починили то, что молча не работало под нагрузкой** (и это, пожалуй, главное), добавили два новых транспорта, довели до конца RAG-петлю для LLM и сделали конкурентность на любом источнике. Ниже — по делу, с кодом.

И сразу главное: с версии 3.3.0 все Pro-решения бесплатны. Никаких лицензий, никаких ключей, никакой регистрации — просто ставишь пакет и юзаешь. Change tracking, bulk, продвинутый кэш, аналитика, кластер Tsak с координатором и failover — всё включено из коробки, в разработке и в проде одинаково. `dotnet add package redb.Postgres.Pro` — и оно работает. Никакого пейволла, никакого лицензионного сервера, ничего активировать не надо.

Цикл про redb и redb.Route. Это продолжение серии, свежие статьи — сверху:

- **redb.Route** — уходим от MassTransit, идём к Apache Camel: Kafka, Scatter-Gather и транзакции
- Apache Camel под .NET: HTTP-коннектор без ASP.NET MVC + Content-Based Router
- **redb.Route** — Apache Camel для .NET, который мы написали потому что выхода другого не было
- **redb** — типизированное хранилище для .NET поверх Postgres/MSSQL: без миграций, без Include, с полным LINQ

Полный список — в профиле. Исходники: github.com/redbase-app. Про саму БД: redb.ru.

Коротко, что за три слоя

Чтобы дальше было понятно, кто есть кто:

- **redb** — типизированное хранилище для .NET. Пишешь обычный POCO-класс, вешаешь атрибут — и работаешь с ним через полный LINQ, серверными запросами, без миграций и без `Include`. Провайдеры: Postgres, MSSQL, SQLite. Есть Free и Pro (change tracking, bulk, кэш, аналитика).
- **redb.Route** — интеграционный движок в духе Apache Camel: DSL из маршрутов (`From( ... ) .... To( ... )`), 30+ коннекторов (Kafka, RabbitMQ, HTTP, gRPC, S3, LLM и т.д.), паттерны интеграции (EIP), транзакции, телеметрия. Тот самый «клей», только не самописный на коленке, а с движком и тестами.
- **redb.Tsak** — рантайм, который берёт маршруты `redb.Route` и превращает их в прод-сервис: дашборд, hot-reload модулей (`.tpkg`), управление контекстами, метрики, кластер с координатором и failover. Поставляется NuGet-пакетами, Docker-образами и standalone-архивами.

Весь код в примерах — на английском, весь текст — по-русски. Всё лежит в репозитории, можно повторять.

Чтобы нижний слой был не только на словах — вот как выглядит `redb` на практике. Обычный класс, атрибут, и дальше полный LINQ серверным запросом, без миграций и без `Include` :

```
[RedbScheme]
public class Note
{
    public string Tag { get; set; } = "";
    public string Text { get; set; } = "";
}

// запись
await redb.SaveAsync(new RedbObject<Note> { Props = new() { Tag = "work", Text = "..." } });

// чтение — серверный запрос, параметр прямо из кода, никакого Include/JOIN руками
var notes = await redb.Query<Note>()
    .Where(n => n.Tag == "work")
    .ToListAsync();
```

Схему создаёт `SyncSchemeAsync<Note>()` — из самого класса, без файлов миграций. Именно этот слой хранит и чанки базы знаний из примера выше, и объекты вашего домена — одинаково.

redb.Route: что нового

Начну с движка — здесь больше всего мяса.

Два новых транспорта: Amazon SQS/SNS и Telegram

redb.Route.Sqs — очередь и топик AWS в один пакет

Один пакет, две схемы: `sqs://` (очередь: консьюмер + продюсер) и `sns://` (топик: publisher + fan-out SNS→SQS). Под капотом — нативный AWS SDK for .NET v4, не самопал.

```
using redb.Route.Sqs.Fluent;

// Консьюмер: 5 конкурирующих воркеров, long-poll, at-least-once (delete on success)
From(Sqs.Queue("orders").WaitTimeSeconds(20).ConcurrentConsumers(5))
    .Process(ex => Handle(ex.In.Body));

// Публикация события в SNS-топик (fan-out в подписанные очереди)
From("timer://tick?period=5000")
    .SetBody(_ => BuildEvent())
    .To(Sns.Topic("order-events"));
```

Объяснить с 

Консьюмер умеет long-polling, `ConcurrentConsumers(N)` (реальные конкурирующие циклы), visibility timeout с опциональным продлением на время обработки, транзакционный ack через `.Transacted()` и FIFO. Продюсер — одиночная и батч-отправка, FIFO group/dedup id. SNS-publisher поддерживает subject, message structure, FIFO и авто-подписку SNS→SQS. Совместимо с LocalStack / ElasticMQ через `serviceUrl=`, полная цепочка AWS-креденшлов, W3C trace-context протаскивается через переход. Подробности — в `redb.Route.Sqs/README.md`.

redb.Route.Telegram — бот как маршрут

Схема `telegram://`, поверх `Telegram.Bot`. Консьюмер — long-polling (`receive`, один `getUpdates` -поток на токен), продюсер —

режимы `send / document / photo / edit / delete / answer`.

```
// Эхо-бот: принял апдейт, ответил в тот же чат
From(Tg.Receive(token))
    .Process(ex => ex.Out.Body = $"Вы написали: {ex.In.Body}")
    .To(Tg.Send(token).ChatId(Header(TelegramHeaders.ChatId)));
```

Объяснить с 

Из коробки — обработка `429 retry_after` (ждёт и повторяет, а не роняет), валидация `parseMode` (кидает на опечатке, а не отправляет сырую разметку молча), `inline/reply`-клавиатуры (`WithInlineKeyboard / WithReplyKeyboard`), `pipeline` распаковки вебхука (`UnpackTelegramUpdate`) и `fluent-DSL` (`Tg.Receive/Send/Document/...`). Доставка — **at-most-once** (Telegram двигает `offset` при выдаче апдейта, это задокументировано), параллелизм — через `.Threads(N)` (см. ниже), телеметрия консьюмера и спаны продюсера.

Два реальных, живых транспорта — `SQS` для AWS-инфраструктуры и `Telegram` для `ChatOps`/ботов. Оба протестированы против эмуляторов (`LocalStack` и мок-сервер).

RAG под ключ: `knowledge://`, `embed://` и `.Knowledge()`

В прошлых версиях у `redb.Route.Llm` уже был универсальный `OpenAI`-совместимый провайдер и нативный `Anthropic`. В 3.3.0 мы **замкнули RAG-петлю** — от загрузки документов до ответа, обоснованного на них, всё внутри маршрутов.

Что появилось:

- **`knowledge://` (ingest-схема)** — продюсер, который берёт тело сообщения как документ, режет на чанки (детерминированные окна по символам с перекрытием), при наличии `IEmbeddingProvider` считает эмбединги и апсертит в `IKnowledgeStore`. Загрузка документов становится маршрутом:

```
From("file://docs?include=*.md").To("knowledge://handbook");
```

Объяснить с 

`Id` чанков — `{docId}#{index}`, так что повторная загрузка того же документа заменяет его чанки на месте.

- **`embed://` (эмбединги как шаг)** — симметрично `llm:// : To("embed://openai")` превращает текст (или коллекцию текстов, порядок сохраняется) в вектор на `Out.Body`. Хост `URI` — имя фабрики

подключения, так что разные маршруты выбирают разные модели эмбедингов по имени.

- **.Knowledge(collection, k) (retrieval-DSL)** — шаг, который достаёт топ-K чанков под текущее сообщение и **вкладывает их в системный промпт**, чтобы следующий `.To("llm://...")` отвечал, опираясь на них:

```
From("kafka://questions")
  .Knowledge("handbook", k: 5)
  .To("llm://claude");
```

Объяснить с 

Семантика, если есть `IEmbeddingProvider` (эмбедем запрос → косинус `SearchAsync`), иначе — ключевой поиск (`SearchTextAsync`, серверный `LIKE` по индексированной колонке). Если стор не подключён или ничего не нашлось — шаг ничего не делает и не ломает pipeline.

- **knowledge_search (готовый инструмент)** — `.AsLlmTool`-маршрут над поиском, чтобы агент сам ходил в базу знаний: вход `{query, top_k?, collection?}` → `{results:[...]}`. `collection` можно **запинить**, тогда аргумент модели игнорируется — так агент запирается в одном тенанте / наборе документов.

Полная петля: `knowledge:// ingest` → эмбединги → ключевой/семантический поиск → инструмент `knowledge_search` или инъекция `.Knowledge()`. Без внешнего векторного стора, если он не нужен: чанки лежат в `redb`, поиск идёт серверным запросом.

Заодно починили неприятную вещь: несколько JSON-сериализаторов LLM-коннектора использовали дефолтный энкодер, который экранирует каждый не-ASCII символ в `\uXXXX`. На результатах инструментов это раздувало число токенов для кириллицы/СJK примерно в 6 раз (и могло всплыть литеральными `\u...` в тексте), а в базе знаний хоронило текст чанка так, что ключевой `LIKE` не мог найти сырой не-ASCII запрос. Перевели на `UnsafeRelaxedJsonEscaping` везде, где надо (`AgentEngine`, `OpenAiProvider`, `McpProtocol`, конверт чанка). Для русскоязычных данных это ощутимо и по деньгам, и по качеству.

.Threads(N) — конкурентность на любом источнике

Появился Camel-style шаг обработки-конкурентности: `From(...).Threads(N)...` `EndThreads()` ограничивает конкурентность участка маршрута числом N. Смысл: даже строго последовательный источник (poll-консьюмер, MQTT, одиночный HTTP-поток) теперь может обрабатывать до N сообщений одновременно — без именованного эндпоинта `seda://`.

```
From("mqtt://sensors")           // источник по природе серийный
    .Threads(8)                   // до 8 обработок параллельно
    .Process(ex => HeavyWork(ex))
    .EndThreads();
```

Объяснить с 

Важная деталь — шаг **адаптивный по паттерну обмена**. Для InOnly (fire-and-forget) это hand-off: клон + пул воркеров, граница транзакции (как `.To("seda://")`). Для InOut тело выполняется **inline на том же exchange под SemaphoreSlim -гейтом**, так что ответ (на Out или In) сохраняется без потерь, request/reply (RPC) работает через шаг, и внешняя транзакция протекает внутрь.

Есть `.MaxQueueSize(n)` и `.EnqueueTimeout(TimeSpan)`. Порядок при $N > 1$ не гарантируется — это цена параллелизма, и она честно задокументирована.

Главное: ConcurrentConsumers(N) наконец действительно работает

А вот теперь — самое важное и самое неприятное, что мы нашли и починили. Держитесь.

ConcurrentConsumers(N) на брокерах молча не давал параллелизма. Опция была, в неё передавали число, оно даже размерило внутренний `SemaphoreSlim` — но реального параллелизма не возникало. Причины были разные в разных коннекторах, а симптом один: консьюмер обрабатывает по одному сообщению за раз, `unacked` растёт до префетча, а маршрут не успевает.

Что конкретно было и что стало:

- **RabbitMQ.** Канал создавался конструктором `CreateChannelOptions(...)` с тремя аргументами, четвёртый (`consumerDispatchConcurrency`) оставался на compile-time дефолте — а в `RabbitMQ.Client 7.2.1` это `1`, не `null`. Ненулевое per-channel значение **перебивает** connection-level настройку, так что каждый канал был зажат в серийную выдачу, а значение с URI/фабрики молча выкидывалось. Теперь `ConcurrentConsumers` — единственный рычаг параллелизма консьюмера: канал всегда открывается с явной `dispatch concurrency`.
(`redb.Route.RabbitMQ 3.2.2.`)
- **Kafka.** Плюс новая опция `EnableAutoCommit` (по умолчанию `true`) — паритет коммита оффсета с `post-process ack` у `RabbitMQ`. И заодно почин транзакционного продюсера, который кидал `Local: Erroneous state` на отложенной отправке.
(`redb.Route.Kafka 3.2.1.`)
- **AMQP 1.0 и IBM MQ.** Тот же класс бага: серийный `receive-loop await` или `Process inline` перед приёмом следующего сообщения, а `SemaphoreSlim` был мёртвым гейтом — его брал и отпускал один и тот же цикл.

Теперь `ConcurrentConsumers(N)` — это **N настоящих конкурирующих консьюмеров**, у каждого своя сессия/соединение и свой цикл (сессии AMQPNetLite и managed-клиент IBM MQ не потокобезопасны, так что параллелизм — от N независимых воркеров, а не от шаринга одного линка). Для IBM MQ это ещё и корректность транзакций: `syncpoint` привязан к соединению, `Backout()` откатывает всё соединение, поэтому каждый воркер обязан владеть своим. Топики IBM MQ при этом зажимаются в одного подписчика с предупреждением — N недолговечных подписок получали бы по копии каждого сообщения, а не делили нагрузку. (`redb.Route.Amqp` / `redb.Route.IbmMq` 3.2.1.)

Все три хотфикса (RabbitMQ 3.2.2, Kafka 3.2.1, `Amqp/IbmMq` 3.2.1) теперь свёрнуты в единый бамп 3.3.0. К каждому — живой регрессионный тест против реального брокера: публикуем пачку сообщений, ставим `ConcurrentConsumers(5)`, проверяем, что наблюдаемая максимальная конкурентность больше 1, а `ConcurrentConsumers(1)` остаётся строго серийным.

На что обратить внимание при обновлении. Если ваш маршрут ставил `ConcurrentConsumers(N > 1)` и «работал» — он работал последовательно. После 3.3.0 он реально параллелится: порядок сообщений на очереди больше не сохраняется на таком маршруте, а его обработчики обязаны быть потокобезопасными. Маршруты с дефолтным 1 не затронуты — как были строго серийными, так и остались.

Плюс в комплекте — новый `AutoAck` у RabbitMQ (broker-side auto-ack / at-most-once, аналог Kafka `EnableAutoCommit`) и почин двойного `BasicAck`, когда `route-level.Transacted()` оборачивает нетранзакционный консьюмер.

Соединения per-exchange: конец «captive singleton»

Второй пласт той же проблемы — уже в ядре движка. `IRedbService` оборачивает одно, непотокобезопасное соединение с БД (модель EF-DbContext). А DSL-шаги `ProcessWithRedb(...)`, `SetBodyFromRedb(...)`, `SetHeaderFromRedb(...)`, `BeginRedbTransaction()` раньше падали в **один** `IRedbService`, захваченный из корневого DI. Под реальной конкурентностью (Splitter с параллелизмом, SEDA, тот самый `ConcurrentConsumers(N)`, параллельные HTTP-запросы) два exchange гнали одно соединение одновременно — и драйвер кидал «*A command is already in progress*» / «*connection is busy*».

Теперь каждый exchange получает **свой DI-scope** → **свой scoped IRedbService** → **своё соединение из пула**, закэшированное на exchange и утилизируемое вместе с ним. Несвязанный синглтон используется только там, где exchange нет (однопоточный сид).

Плюс — `controller.Redb()` для контроллеров: резолвит per-request инстанс вместо общего captive-синглтона.

Заодно закрыли утечки DI-scope/соединений на жизненном цикле `exchange: ThreadsProcessor`, `SedaProducer / VmProducer` (утекал клон при неудачном hand-off), `WireTapProcessor` (утекал tap-клон при исключении в `onPrepare / newBody`), `scheduled`-консьюмеры `llm://` и `exec://` (не утилизировали per-tick exchange). Всё теперь диспозится в `finally`. И сделали ленивый старт продюсера потокобезопасным — на холодном старте под конкурентностью можно было поймать `NullReferenceException` (например, Redis-продюсер с ещё не инициализированным `_db`); теперь оба пути `single-flight`, продюсер виден как стартовавший только после полного `ConnectAsync()`.

Если коротко: 3.3.0 — это релиз, после которого `redb.Route` действительно можно грузить конкурентно и не ловить плавающие ошибки соединений. Именно это и отличает «работает на демо» от «работает в проде».

Собираем всё вместе: RAG-бот в Telegram за несколько маршрутов

Чтобы новые фишки не выглядели списком, покажу, как они складываются в одну вещь. Задача: Telegram-бот, который отвечает по вашей базе знаний (набор markdown-файлов), а не выдумывает. Три маршрута — загрузка знаний, приём вопроса и ответ, и всё это на конкурентном источнике.

Сначала — загрузка документов в базу знаний. Один маршрут: читаем файлы, режем на чанки, эмбеддим, кладём в стор. Стор — `redb`, отдельный векторный движок не нужен.

```
// 1. Ingest: папка markdown → чанки с эмбедами в redb
From("file://docs?include=*.md")
    .To("knowledge://handbook?chunkChars=1000&overlap=100&embed=true");
```

Объяснить с 

Теперь сам бот. Принимаем апдейт из Telegram, достаём топ-5 релевантных чанков под текст вопроса, вкладываем их в системный промпт и спрашиваем модель. `.Knowledge()` сам решит — семантика (если подключён `IEmbeddingProvider`) или ключевой поиск.

```
// 2. Вопрос из Telegram → RAG → ответ обратно в тот же чат
From(Tg.Receive(token))
    .Threads(4) // до 4 диалогов параллельно
    .Knowledge("handbook", k: 5) // топ-5 чанков в системный промпт
```

```
.To("llm://claude") // ответ, обоснованный на них
.EndThreads()
.To(Tg.Send(token).ChatId(Header(TelegramHeaders.ChatId)));
```

Объяснить с 

Вот и всё. Обратите внимание, чего здесь нет: нет ручной работы с векторами, нет отдельного сервиса эмбедингов, нет самописного клея между Telegram и LLM, нет пула соединений, за которым надо следить. `.Threads(4)` даёт параллельную обработку диалогов на источнике, который сам по себе выдаёт апдейты по одному; каждый параллельный поток при обращении к `redb` получит своё соединение.

Если хочется, чтобы модель сама решала, когда лезть в базу знаний (а не жёстко на каждый вопрос), — вместо `.Knowledge()` даём агенту инструмент `knowledge_search` и пинуем коллекцию, чтобы он не мог выйти за пределы `handbook` :

```
// регистрируем инструмент один раз; коллекция запинена — агент заперт в ней
context.AddRoutes(new KnowledgeSearchTool(new KnowledgeSearchOptions {
    Collection          = "handbook",
    EmbeddingProvider = embeddings // есть → семантика; нет → ключевой по
}));
```

```
From(Tg.Receive(token))
    .Threads(4)
        .To("llm://claude?tools=knowledge_search")
    .EndThreads()
    .To(Tg.Send(token).ChatId(Header(TelegramHeaders.ChatId)));
```

Объяснить с 

Один и тот же набор возможностей

— `knowledge://`, `embed://`, `.Knowledge()`, `knowledge_search`, `telegram://`, `.Threads()` — и всё это появилось или дозрело в 3.3.0. По отдельности каждая штука небольшая; вместе — это RAG-бот в проде без единой внешней зависимости сверх модели.

redb (ядро БД): что вошло

В ядре — три почина и один защитный механизм, и все они в той же теме конкурентности и соединений.

- **Fail-fast guard на соединении провайдера** (`RedBase.Postgres` , `RedBase.MSSql` , `RedBase.SQLite + .Pro`). Если один и тот же `IRedbService` входят из двух потоков сразу, провайдер теперь кидает понятный `InvalidOperationException` с указанием причины — вместо мутной ошибки драйвера («*A command is already in progress*», «*connection is busy*», «*another read operation is already in progress*»). Лёгкая `Interlocked` -проверка, нулевая цена на нормальном однопоточном пути. Это дополняет `per-exchange` соединения в `redb.Route`: если кто-то всё-таки шарит инстанс — узнаёт об этом сразу и по-человечески.
- **Парсер запросов: `array.Contains(x)` в `WhereRedb` падал на .NET 9 / C# 13.** `string[]` (или любой массив) `.Contains(x)` внутри предиката теперь биндится к `overload'y ReadOnlySpan ( MemoryExtensions.Contains )`, а не `Enumerable.Contains`, который парсер фильтра отвергал с `NotSupportedException`. Парсер распознаёт `MemoryExtensions.Contains`, разворачивает конверсию массив→спан и переводит в тот же `IN`, что и `Enumerable.Contains`. Мелочь, но на новом компиляторе ловилась на ровном месте.
- **`ComputeHash()` NRE на объекте с `Props == null`.** Обобщённый путь `ComputeFor<TProps>` разыменовывал объект до `null`-проверки. Добавили `guard` — путь консистентно возвращает `null` (→ `Guid.Empty`) вместо исключения.
- **Утечка соединений из пула на `dispose` транзакции/соединения** (`RedBase.Postgres` , `RedBase.MSSql` , `RedBase.SQLite + .Pro`). Бросок из `DisposeAsync()` транзакции драйвера (возможен в разгар «шторма ошибок» на уже сломанном соединении) проскакивал мимо `dispose` самого `_connection` — физическое соединение не возвращалось в пул. А так как `SaveAsync` идёт в явной транзакции, каждая запись взводила этот путь: под всплеском ошибок утечка сама себя усиливала и в итоге вычерпывала пул (симптом: здоровый пул внезапно перелезает за `MaxPoolSize` с таймаутами, лечится только рестартом). Теперь `dispose` соединения и транзакции идут через `try/finally` — соединение возвращается всегда, а сам фолт диспоза не глотается, а пробрасывается, чтобы оставался наблюдаемым.

redb.Tsak (рантайм): что вошло

Рантайм в 3.3.0 подтягивает всё вышеперечисленное и добавляет своё.

- **Новые коннекторы в комплекте.** `redb.Route.Sqs` (`sqs://` , `sns://`) и `redb.Route.Telegram` (`telegram://`) теперь входят в дистрибутив — модули

на Tsak могут их использовать без ручной доукомплектации. Технически это правка shared-слоя сборки (коннекторы кладутся в `Libs/shared`), и — спойлер из внутренней кухни — именно на этом шаге мы поймали, что скрипт сборки образа не знал про новые коннекторы: собрали образ локально, подняли контейнер, увидели в логах `Loaded shared assembly redb.Route.Sqs` и `redb.Route.Telegram` — только после этого публиковали.

- **Дашборд-планировщик показывает cron-задачи даже без секции Quartz**. Раньше без конфига Quartz Tsak не регистрировал общий `IScheduler`, cron-маршрут сам себе поднимал *per-context* in-memory планировщик, которого не видел управляющий `_system`-контекст — и страница планировщика была пустой, хотя маршрут работал. Теперь Tsak всегда выдаёт один общий `IScheduler` (fallback на in-memory `RAMJobStore`, если секции нет). Для одного узла не нужен ни кластер, ни база; `AdoJobStore` остаётся способом персистить и шарить задачи между узлами.
- **Users admin API — per-request scoped IRedbService**. `UserController` раньше резолвил общий captive-синглтон, и параллельные админ-запросы могли конфликтовать. Теперь — per-request инстанс через `controller.Redb()`.
- **Из 3.2.1** (свёрнуто сюда): страница Endpoints больше не прячет анонимные контексты, которые при этом считала в статистике; standalone-архив web-дашборда стартует на задокументированном порту 8080, а не на дефолтном 5000.

Публикуется, как обычно, тремя способами: NuGet-пакеты, Docker-образы `redb-tsak-{worker,web,stack}` (.NET 9, теги `:3.3.0-net9`, `:3.3.0`, `:latest`) на ghcr.io/redbase-app с cosign-подписью, и standalone-архивы `redb-tsak-3.3.0-linux-x64.tar.gz` / `redb-tsak-3.3.0-win-x64.zip` с `checksums.txt` и `.bundle`-подписями на GitHub-релизе. Всё на **.NET 9**.

[СКРИН: дашборд Tsak на localhost:8080 — список контекстов с системным echo-маршрутом]

Про бесплатный Pro — ещё раз, подробнее

Я сказал это во вводной, но повторю, потому что вопрос «а в чём подвох» возникает первым. Подвоха нет. Pro остаётся проприетарным (исходники закрыты), но на всей линейке 3.x раздаётся **даром и без всякого учёта** — нет лицензий, ключей, регистрации, лицензионного сервера или лимитов по узлам/объёму.

Что именно раньше было платным, а теперь бесплатно: у ядра `redb` — `change tracking`, `bulk`-операции, продвинутый кэш, аналитические запросы (группировки, оконные функции); у

Tsak — кластер с координатором, распределённый лок, leader election, failover. То есть ровно то, за что в других стеках берут деньги. Здесь — dotnet add package и работаете.

Обновление с 3.2.x: короткий чек-лист

Релиз обратно совместим по API, но есть один поведенческий момент, который стоит проверить — тот самый параллелизм.

- **Пройдитесь по маршрутам с ConcurrentConsumers($N > 1$)**. Раньше они работали серийно (баг), теперь — реально параллельно. Убедитесь, что обработчики на таких маршрутах потокобезопасны и что вы не полагаетесь на порядок сообщений в очереди. Если порядок критичен — оставьте ConcurrentConsumers(1) (это дефолт, поведение не изменилось).
- **Проверьте разделяемое состояние в обработчиках**. Всё, что раньше «случайно работало», потому что маршрут был де-факто однопоточным, теперь может выполняться из N потоков.
- **IRedbService в маршрутах — ничего делать не надо**. Per-exchange соединения включились сами; наоборот, пропадут прежние «*A command is already in progress*» под конкурентностью. Если вы вручную кэшировали один IRedbService на несколько exchange в обход DSL — перестаньте, ядро теперь про это сообщит fail-fast.
- **Бамп версий — единый**. Все пакеты экосистемы едут на 3.3.0 ; смешивать redb.Route 3.3.0 с коннектором 3.2.0 не нужно (хотя старые коннекторы 3.2.x совместимы по API — они просто без свежих фиксов конкурентности).
- **Tsak: пересоберите образ/возьмите новый**. Если гоняете свою сборку дистрибутива — новые коннекторы (Sqs , Telegram) попадают в shared-слой только после пересборки; готовые образы :3.3.0 уже с ними.

Ничего из этого не срочно и не ломает сборку — это про «посмотреть, где у вас реально включился параллелизм».

Как поставить

Ядро и провайдеры — с NuGet:

```
dotnet add package redb.Core
dotnet add package redb.Postgres      # или redb.MSSql / redb.SQLite
dotnet add package redb.Postgres.Pro  # Pro — бесплатно, без ключа
```

Объяснить с 

Движок и нужные коннекторы:

```
dotnet add package redb.Route
dotnet add package redb.Route.Sqs
dotnet add package redb.Route.Telegram
dotnet add package redb.Route.Llm
```

Объяснить с 

Рантайм — образом или архивом:

```
docker pull ghcr.io/redbase-app/redb-tsak-stack:3.3.0
# либо standalone-архив с GitHub-релиза v3.3.0 (linux-x64 / win-x64), с cosign-
```

Объяснить с 

Образы подписаны cosign, публичный ключ — в релизе (cosign.pub):

```
cosign verify --key cosign.pub ghcr.io/redbase-app/redb-tsak-worker:3.3.0
```

Объяснить с 

Итог

3.3.0 — это про зрелость. Новые транспорты (SQS/SNS, Telegram) и завершённый RAG — приятно, но главное в этом релизе не они. Главное — что мы нашли и починили **системную дыру в конкурентности**: ConcurrentConsumers(N) молча не параллелил ни на одном брокере, а общий захваченный IRedbService разваливался под нагрузкой мутными ошибками соединений. Это ровно тот класс багов, который не видно на демо и который выстреливает в проде под нагрузкой. Теперь конкурентность честная — на брокерах, на любом источнике через .Threads(N) , и с соединениями per-exchange.

Плюс — весь стек синхронно на 3.3.0, всё на .NET 9, Pro бесплатен без ключа. Своя БД, свой интеграционный движок, свой рантайм с дашбордом и кластером — как одна экосистема, а не зоопарк.

Если попробуете — пишите в комментариях, что ловится в вашем сценарии. По конкретным коннекторам и паттернам EIP выходят отдельные разборы в серии.

Исходники и релизы: github.com/redbase-app. Про БД redb: redb.ru. Прошлые статьи — в профиле.

If this was useful — a 🌟 on GitHub helps others find it.

Теги: `c#`, `sqlite`, `dotnet`, `kafka`, `rabbitmq`, `redb`, `sqs`, `telegram`

Хабы: `.NET`, `C#`, `Open source`, `SQLite`

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



32K+

Охват за 30 дней

4

Карма

Kozin Rinat @grelikt

Пользователь

Подписаться



 Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



Shannon

23 часа назад

habrGPT. Обучим LLM 0.5B с нуля на статьях Хабра с помощью nanochat от Карпатого. Обучение fp8 дома и сравнение с bf16



17 мин



18K



+107



165



39



wmlab

17 часов назад

Планета, которой не было



Простой



13 мин



14K



+40



9



12



kaftanati

19 часов назад

Умный летний дом своими руками (буквально)



Простой



20 мин



14K

Ретроспектива



+39



44



23



Li_Lu09

19 часов назад

«Авторский огонёк» Хабра: почему мы зажигаем авторские сердца



Простой



21 мин



9.5K

 +32

 5

 9



Flampanzer
9 часов назад

Управление ключами SSH — вызовы и эволюция подходов

 9 мин  7.7K

Обзор

 +26

 31

 2



ProstoKirReal
9 часов назад

Сложно о простом. Все, что бы вы хотели знать о SFP модулях. Часть 6. Топ-10 частых ошибок при выборе и эксплуатации SFP

 Средний  9 мин  9.8K

Обзор

 +25

 42

 9



GlinkinIvan
5 часов назад

Хакер в магазине: изучаем поверхность атаки типичного супермаркета

 9 мин  6.8K

 +23

 18

 3



arturdumchev
6 часов назад

Калвиаутра путает калвиши: один байт изменил всё

 Средний  8 мин  5K

Тutorial

 +21

 3

 0



alexeybashuk
9 часов назад

Нейросети: кому на самом деле принадлежат права на то, что вы там генерируете — разбираемся по-человечески

 Простой  9 мин  6.1K

Аналитика

 +20

 12

 15



DAN_SEA

8 часов назад

Про плазменный электролиз — как перспективный способ добычи кислорода и водорода из воды

 Простой  12 мин  3.3K

Обзор

 +19

 8

 1

Суперсилы и слабости IT-компаний: началось исследование работодателей 2026

Турбо

Показать еще

ИСТОРИИ



Унеси с Робозона свои миллионы



Пришёл, смастерил, рассказал — сезон DIY в разгаре



От лунного овоща до сверхострого чили



Годнота из блогов компаний



Из центра Млечного Пути



Тол...

ВОПРОСЫ И ОТВЕТЫ

Почему разрывается подключение к бд на сервере?

Python · Простой · 1 ответ

Как сделать правильную перемотку видео в Flyleaf (wpf)?

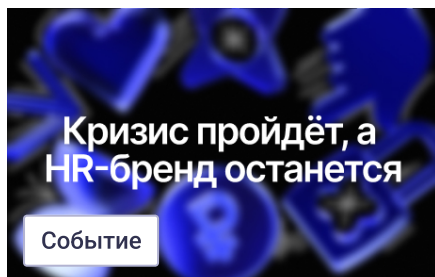
Видео · Сложный · 1 ответ

Как передать изображение на принтер TSC?

C# · Средний · 1 ответ

Больше вопросов на Хабр Q&A

МИНУТОЧКУ ВНИМАНИЯ



Бесплатный вебинар о том, как строить HR-бренд в 2026 году



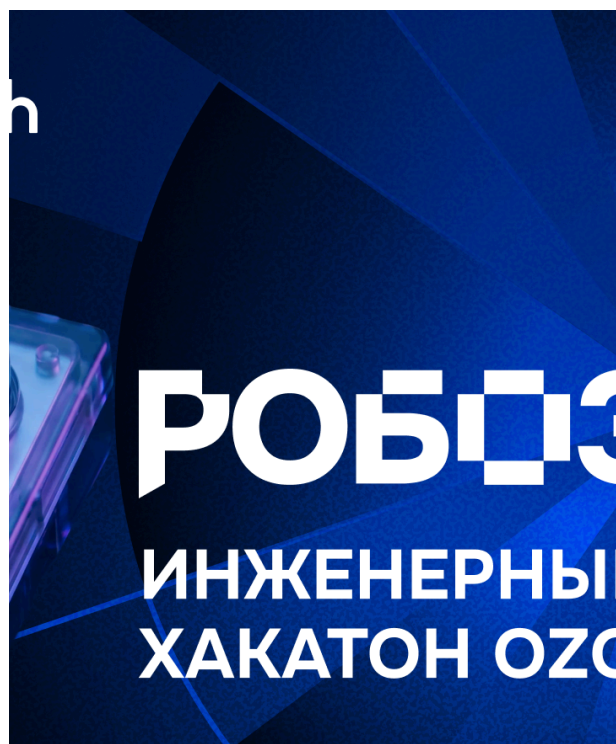
Турбо

DPL в Overlay-слое: общение виртуальных машин

Турбо

Просто добавь IT: как совмещать код и свёрла в сезоне DIY

БЛИЖАЙШИЕ СОБЫТИЯ



2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная премия,
учрежденная в 2000 году**

Санкт-Петербург

Маркетинг

Больше событий в календаре

11 июня – 2 сентября

**Робозон: хакатон с призовым
фондом 15 млн руб.**

Москва • Онлайн

Разработка

Больше событий в календаре

15 июля

**Веби
эпох**

Онлайн

Маркетинг

Больше событий в календаре



Настройка языка

Техническая поддержка

